1. 알고리즘 효율 CSED331 · 알고리즘 전체 홈↗

이 페이지

1. 무엇을 비교하는가
2. Fibonacci: FIB1 (순진한 재귀)
3. FIB2: 값을 저장해서 재사용 — bottom-up DP (tabulation)
4. 성장률 (growth rate)
5. Convex Hull
6. 점근 표기 (asymptotic notation): O, Ω, Θ, o, ω
7. 연습문제

알고리즘 효율 (efficiency)

2026-09-07 (월) · 슬라이드 Computational Efficiency (p.1–22, 녹취 없음, 교수 marked PDF)

무엇을 비교하는가

맞는가 (correct)	올바른 답을 내는가
시간은 얼마인가	입력 크기 n의 함수 $T(n)$으로 표현
더 잘할 수 있나	같은 문제를 더 빠르게 푸는 방법이 있나
왜 초(second)가 아니라 연산 수를 세나	같은 코드도 컴퓨터마다 시간이 다름 → 기계 의존적 연산 수는 실제 실행 시간보다 기계의 세부 사항에 덜 의존하는, 알고리즘 자체의 특성
왜 입력 크기 n의 함수로 보나	입력이 작으면 점근적 성장률의 차이가 덜 드러나는 경우가 많지만, 모든 알고리즘이 반드시 빠른 것은 아니다 입력이 커질수록 알고리즘의 질이 갈림 → **큰 n에서의 추세**가 본질
1 단위	상수 시간 $O(1)$ 연산(덧셈, 비교, 배열 접근 …)을 **1 단위**로 셈

반복문 세기

- for i = 1..n에서 안쪽 연산이 상수 개면 기본 연산 수는 n.
- 안쪽에 priority queue delete-min이 있으면 한 번에 $O(\log n)$이므로 전체 $O(n \log n)$. 반복 횟수 × 안쪽 연산 수로 센다.

Fibonacci: FIB1 (순진한 재귀)

피보나치 수

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \ (n \geq 2)$$

음수 입력 검사 같은 예외 처리는 여기선 논외로 함

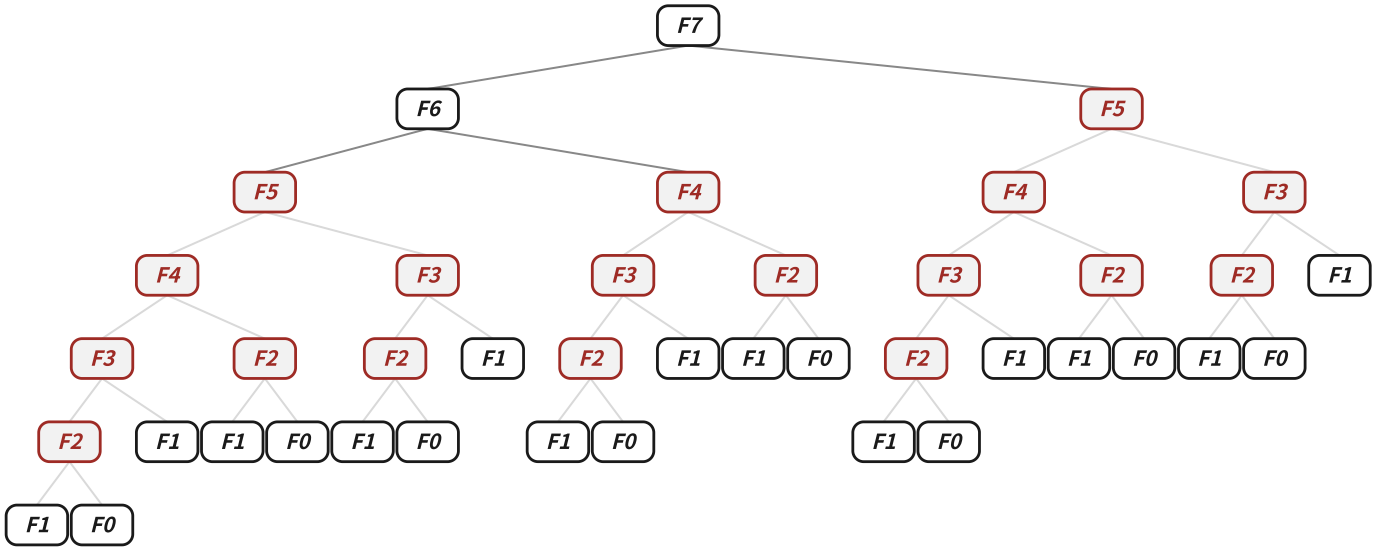
FIB1 — 정의를 그대로 재귀로 옮김

```
FIB1(n)
    if n = 0: return 0
    if n = 1: return 1
    return FIB1(n-1) + FIB1(n-2)
```

$$T(n) \leq 2 \ (n \leq 1), \qquad T(n) = T(n-1) + T(n-2) + O(1) \ (n > 1)$$

- $T(n)$ = FIB1(n)의 **총 호출 횟수** (호출 1번 = 1로 셈, $T(0) = T(1) = 1$)
- 점화식이 피보나치와 같은 모양 → 귀납으로 $T(n) \geq F_n$
  - 기저: $T(0) = 1 \geq F_0 = 0, T(1) = 1 \geq F_1 = 1$
  - 귀납: $T(n) = T(n-1) + T(n-2) + 1 \geq F_{n-1} + F_{n-2} + 1 > F_n$

호출 트리와 중복 계산



FIB1(7)의 호출 트리 — **빨간 노드 = 중복 계산**. F5는 2번, F4는 3번, F3는 5번, F2는 8번 계산된다

- F_k의 호출 횟수 = F_{n-k+1} $(k \geq 1)$, F_0의 호출 횟수 = F_{n-1}
- 1을 반환하는 잎 F_n개 + 0을 반환하는 잎 F_{n-1}개 = 전체 잎 F_{n+1}개
- 내부 노드(덧셈하는 호출)는 $F_{n+1} - 1$개 ($n = 7$이면 20개)

손계산 1 — $T(n)$과 덧셈 수 표

n	F_n	$T(n)$ (호출 수)	덧셈 수
0	0	1	0
1	1	1	0
2	1	3	1
3	2	5	2
4	3	9	4
5	5	15	7
6	8	25	12
7	13	41	20

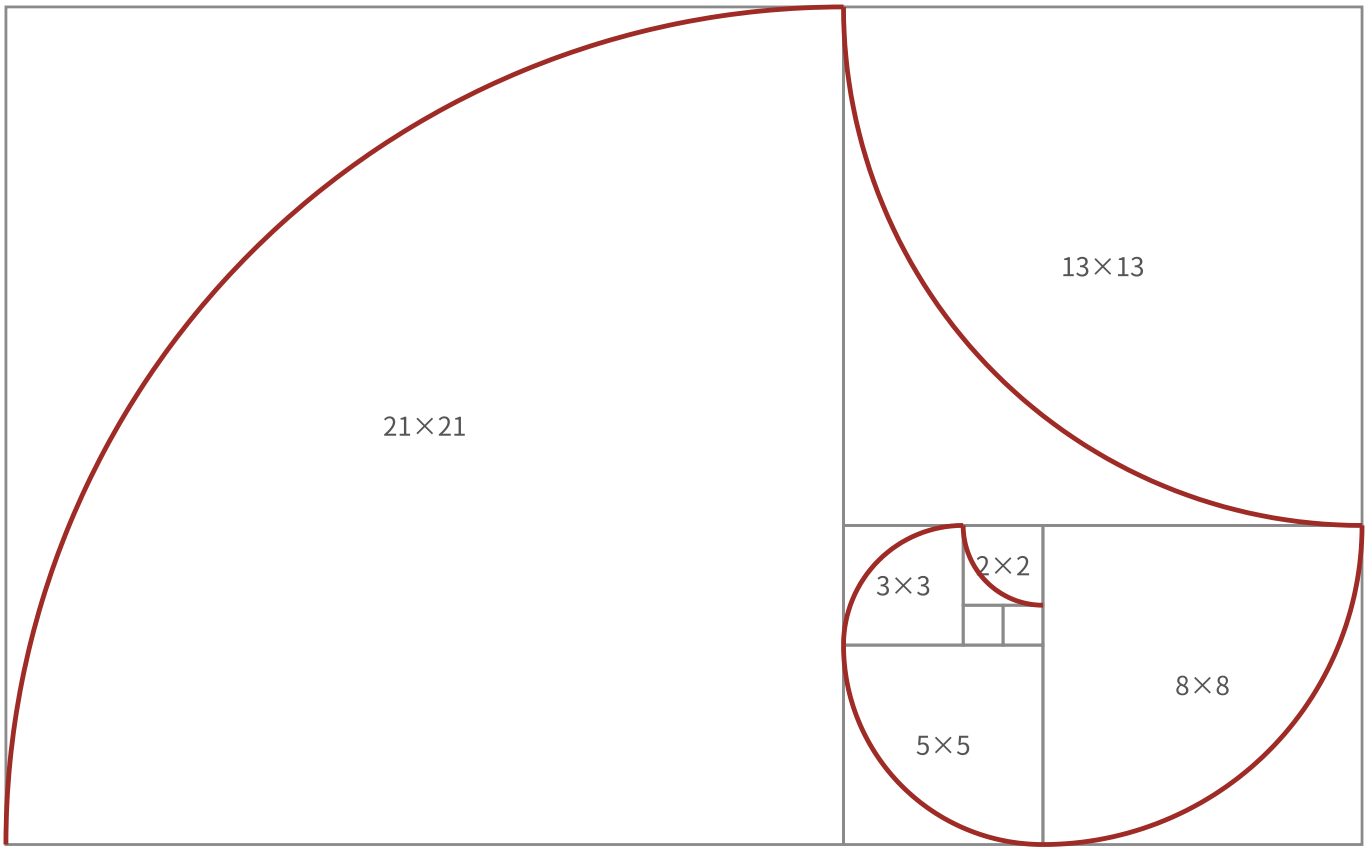
8	21	67	33
9	34	109	54
10	55	177	88

- 표에서 규칙 발견: $T(n) = 2F_{n+1} - 1$, 덧셈 수 = $F_{n+1} - 1$
- 검산: $T(7) = 2F_8 - 1 = 2 \times 21 - 1 = 41$ ✓

FIB1의 시간은 지수적

피보나치 수는 F_n ~ φⁿ/√5이며, φ = (1+√5)/2 ≈ 1.61803398887이다. FIB1의 총 호출 횟수는 T(n) = 2F_{n+1} − 1이므로 정확한 점근적 성장률은 T(n) = Θ(φⁿ)이다. 슬라이드에서 사용하는 1.6ⁿ은 수치 추정을 위한 근사일 뿐이며, φ ≠ 1.6이므로 T(n) = Θ(1.6ⁿ)이라고 점근적 동치를 나타내서는 안 된다.

그림 · 피보나치 나선 (황금비 수렴)



피보나치 나선 — 변 5, 8, 13, 21, 34의 이웃 비율 1.6, 1.625, 1.615, 1.619, … 는 황금비 φ로 수렴

- 1.6^n 근사 vs 정확한 덧셈 수 $F_{n+1} - 1$:

n	**슬라이드 근사 1.6^n**	**정확한 덧셈 수**
10	≈ 110	88
20	≈ 12,089	10,945
50	≈ 1.6×10^{10}	≈ 2.0×10^{10}
100	≈ 2.6×10^{20}	≈ 5.7×10^{20}

$T(n) \neq \Theta(2^n)$ **이다!**

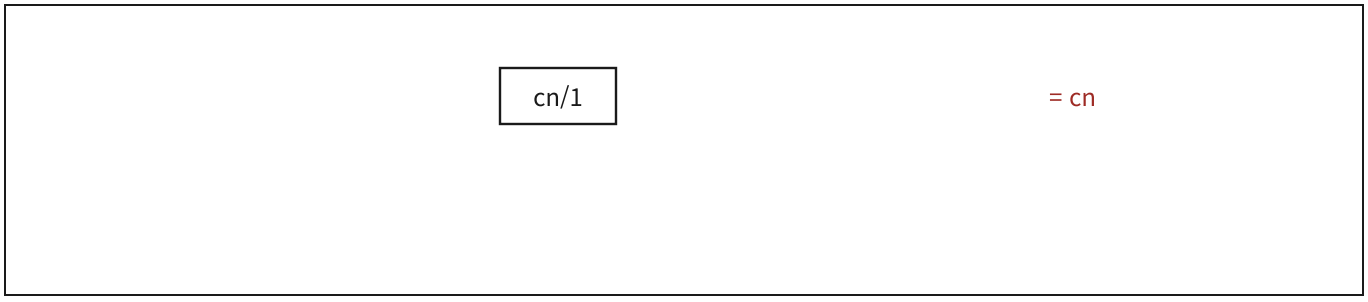
- "지수니까 2^n"라고 쓰고 싶어지는 대표적 실수
- $\varphi < 2$ 이므로 $T(n)/2^n \to 0 \to T(n) = O(2^n)$ 이고 $o(2^n)$ 이지만 $\Theta(2^n)$은 아님

n	$T(n)$	2^n	$T(n)/2^n$
10	177	1,024	0.173
30	2,692,537	≈ 1.07 × 10⁹	0.00251
50	≈ 4.07 × 10¹⁰	≈ 1.13 × 10¹⁵	3.62 × 10⁻⁵

원문 계산 예: FIB1의 실제 시간

- n이 10 늘면 덧셈 수는 약 123배 ($\varphi^{10} \approx 123.0$).
- 초당 2.5×10^{14}회 연산: FIB1(100)의 슬라이드 어림 1.6^{100}은 약 12일. 정확한 덧셈 수 $F_{101} - 1 \approx 5.73 \times 10^{20}$이면 약 26.5일.
- FIB1(200): 호출 $2F_{201} - 1 \approx 9.1 \times 10^{41}$번, 같은 속도에서 약 1.2×10^{20}년. 우주 나이 약 1.4×10^{10}년의 약 10^{10}배.
- FIB1(1000): 호출 $2F_{1001} - 1 \approx 1.4 \times 10^{209}$번.

재귀 트리: 병합 정렬의 점화식



T(n) = 2T(n/2) + cn. 각 노드는 부분 문제의 비재귀 작업량.

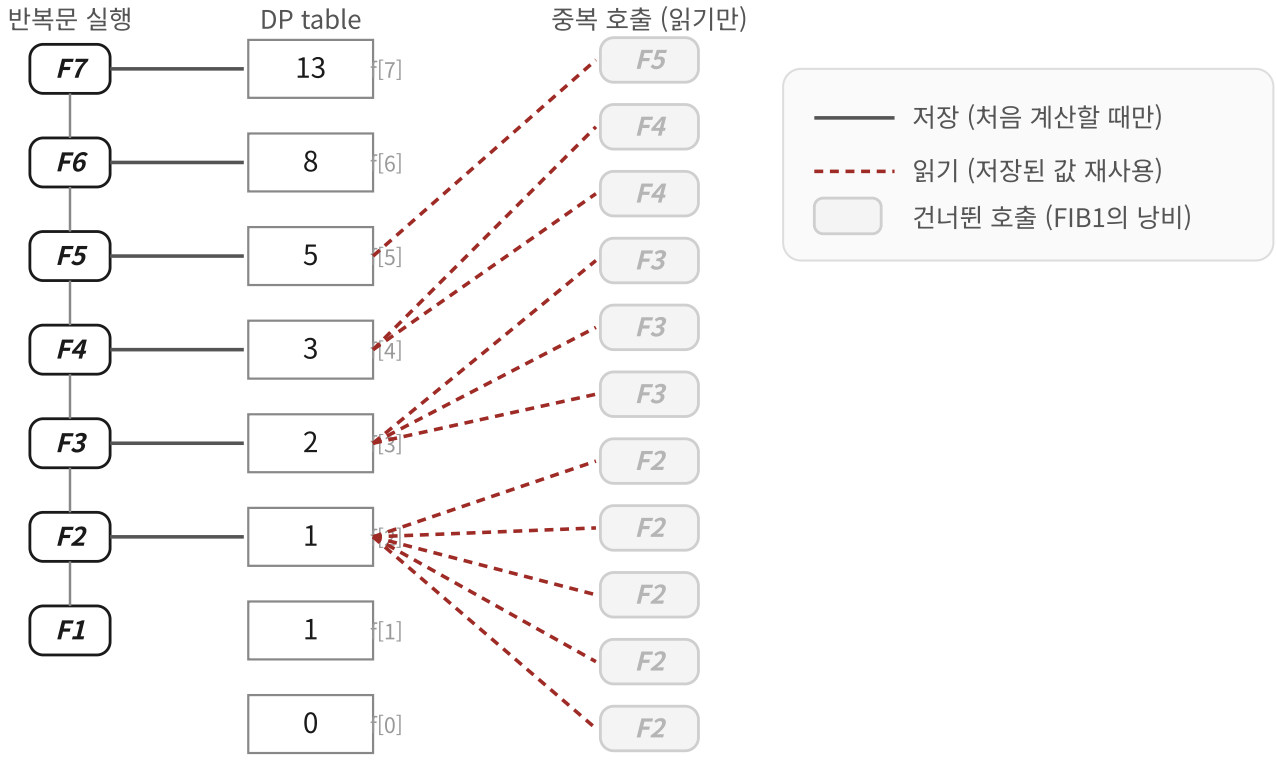
◀ 이전 1 / 4 다음 ▶

FIB2: 값을 저장해서 재사용 — bottom-up DP (tabulation)

> **한 번 계산한 F_i는 배열 f에 저장하고, 필요할 때 다시 계산하지 않고 읽어 옴. 값을 한 번만 계산해 저장·재사용한다는 점은 재귀에 캐시를 붙인 memoization과 같지만, FIB2는 재귀 호출 없이 작은 값부터 반복문으로 표를 채우는 tabulation(bottom-up DP)이다**

FIB2

```
FIB2(n)
    if n = 0: return 0
    array f[0..n]
    f[0] = 0; f[1] = 1
    for i = 2..n: f[i] = f[i-1] + f[i-2]
    return f[n]
```



FIB2(7) — 반복문 실행이 한 줄로 진행되고, 각 값은 한 번만 계산(저장)된 뒤 **읽기**로 재사용. 흐린 노드 = FIB1이 낭비하던 중복 호출

손계산 2 — FIB2(7) 추적과 연산 수

i	$f[i-1]$	$f[i-2]$	$f[i]$

2	1	0	1
3	1	1	2
4	2	1	3
5	3	2	5
6	5	3	8
7	8	5	13

- 연산 수 세기: 비교($n = 0$) 1 + 배열 생성 $n + 1$ + 대입 2 + 반복 $n - 1$회 × 상수(반복당 6)
- 합계 $(n + 1) + 1 + 2 + 6(n - 1) = 7n - 2 \rightarrow$ $T(n) = O(n)$

n	**FIB1 덧셈 수**	**FIB2 덧셈 수**
10	88	9
20	10,945	19
30	1,346,268	29
50	≈ 2.0×10^{10}	49

- "배열 접근 = $O(1)$"은 이 분석의 **가정**: F_{100}은 21자리, F_{1000}은 209자리 수라 실제 큰 수 덧셈은 상수가 아님. 여기서는 **연산 횟수**만 센다는 관점임을 기억. 단위 비용 RAM 모델에서 덧셈·비교·배열 접근을 O(1)로 가정하면 FIB2는 Θ(n)번의 기본 연산을 수행한다. 임의 정밀도 정수의 비트 연산으로 분석하면 시간 복잡도는 $O(n^2)$이다

- 보통 polynomial time이면 efficient라고 부른다. FIB2보다 빠른 방법도 존재하므로 "더 잘할 수 있나"라는 질문은 계속 남는다.

성장률 (growth rate)

함수별 실제 시간 감각

	n	$n \log n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1s	< 1s	< 1s	< 1s	< 1s	< 1s	seconds
$n = 30$	< 1s	< 1s	< 1s	< 1s	< 1s	minutes	10^{19} yrs
$n = 100$	< 1s	< 1s	< 1s	< 1s	10K yrs	10^{17} yrs	—
$n = 1{,}000$	< 1s	< 1s	< 1s	minutes	—	—	—
$n = 10^6$	≈ 1s	seconds	days	30K yrs	—	—	—

슬라이드 표 (— = 표에서 사실상 계산 불가로 표시, n! 칸은 초당 10^6 연산 가정으로 정정). 이론적 분석에서는 다항 시간 알고리즘을 일반적으로 효율적인 알고리즘으로 분류한다. 다만 실제 실행 가능성은 다항식의 차수, 상수 계수, 메모리 요구량, 입력 크기 등에 따라 달라진다.

$$\log n \ < \ n \ < \ n \log n \ < \ n^2 \ < \ n^3 \ < \ 1.5^n \ < \ 2^n \ < \ n!$$

큰 n 기준의 대소. **작은 n에서는 뒤집힐 수 있음**: $n = 10$에선 $1.5^n = 57.7 < n^3 = 1000$ 이지만 $n = 30$ 이면 $1.5^n \approx 191{,}751 > n^3 = 27{,}000$

$\log n$이 얼마나 작은가

- 암산 도구: $2^{10} = 1024 \approx 10^3 \rightarrow \log_2 1000 \approx 10$
- $\log(ab) = \log a + \log b$ 이므로 $\log_2 10^6 \approx 20, \log_2 10^{12} \approx 40$
- n이 백만 배 커져도 $\log n$은 겨우 두 배:

n	$\log_2 n$	$n \log_2 n$	n^2
10^3	9.97	9.97×10^3	10^6
10^6	19.93	1.99×10^7	10^{12}

10^{12}	39.86	3.99×10^{13}	10^{24}

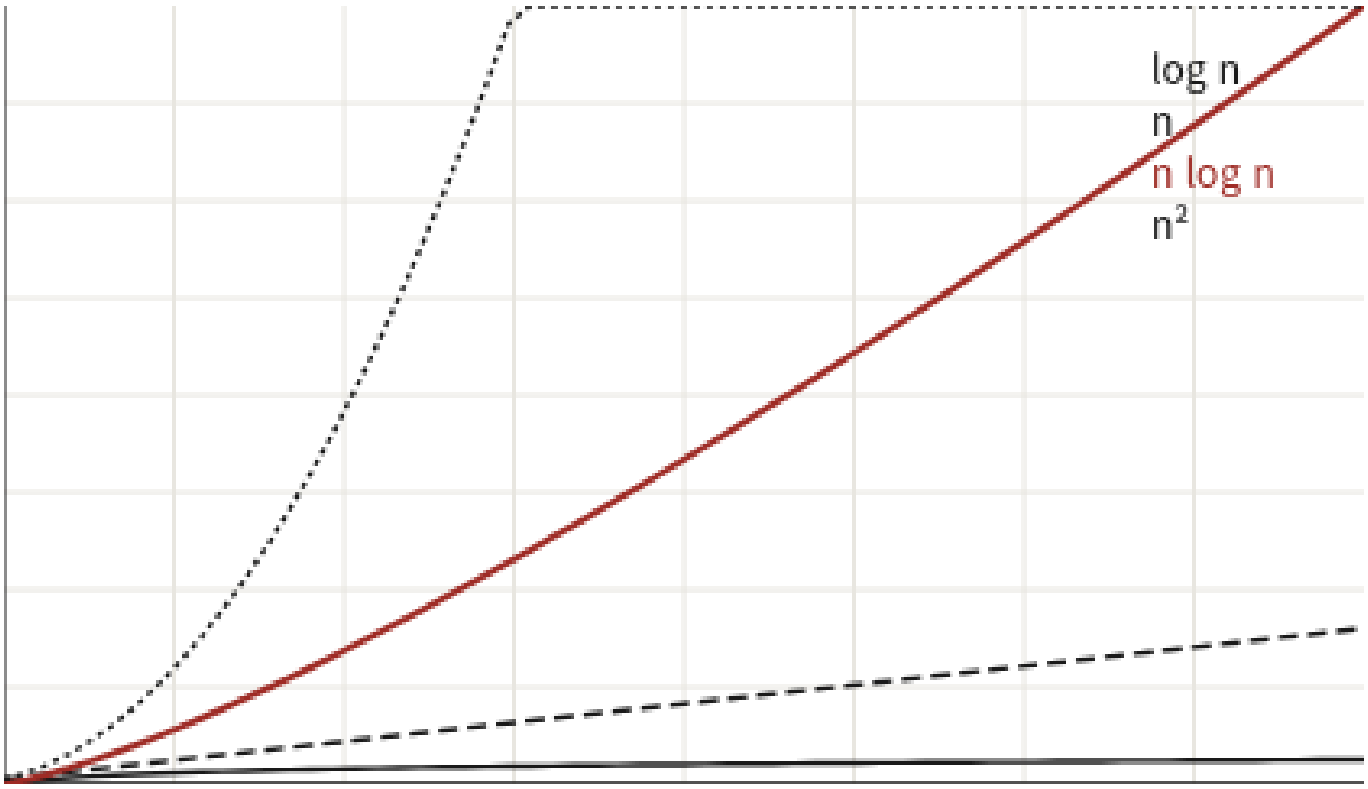
- $n \log n$은 n^2보다 n에 훨씬 가까움 → $O(n \log n)$ 정렬(quicksort 등)은 $n = 10^6$에서도 대체로 빠르게 동작한다. 다만 실제 실행 시간은 입력 크기와 구현에 따라 달라지며, quicksort의 최악 시간은 O(n²)이다
- **polylog는 polynomial에 눌림**: $n^2 \log n < n^3$, $n \log^3 n < n^2$

손계산 3 — $n = 10^6$, 초당 10^6 연산이면

- n : 10^6 회 → **1초**
- $n \log_2 n$: $\approx 2 \times 10^7$ 회 → **20초**
- n^2 : 10^{12} 회 → 10^6초 $\approx$ **11.6일**
- n^3 : 10^{18} 회 → 10^{12}초 $\approx$ **3만 년**
- n^2 대 $n \log_2 n$의 비: $n^2/(n \log_2 n) \approx 5.0 \times 10^4$ 배

원문 보충: 성장률의 경계와 표 재계산

- 초당 10^6회: 2^{30}은 17.9분, 1000^3은 16.7분, 10!은 3.6초. 1.5^{100}은 약 1.29×10^4년, 2^{100}은 약 4.02×10^{16}년.
- $100^3 = 10^6$은 정확히 1초. 30!은 약 2.65×10^{32}이므로 약 8.4×10^{18}년. 슬라이드의 10^{25} yrs는 이 연산 속도 가정과 맞지 않음.
- $1.5^n > n^3$이 끝까지 성립하는 첫 정수는 n = 24. $n > \log_2^3 n$은 n = 982부터 끝까지 성립. dominates는 점근적 사실이며 작은 n에서는 반대일 수 있음.



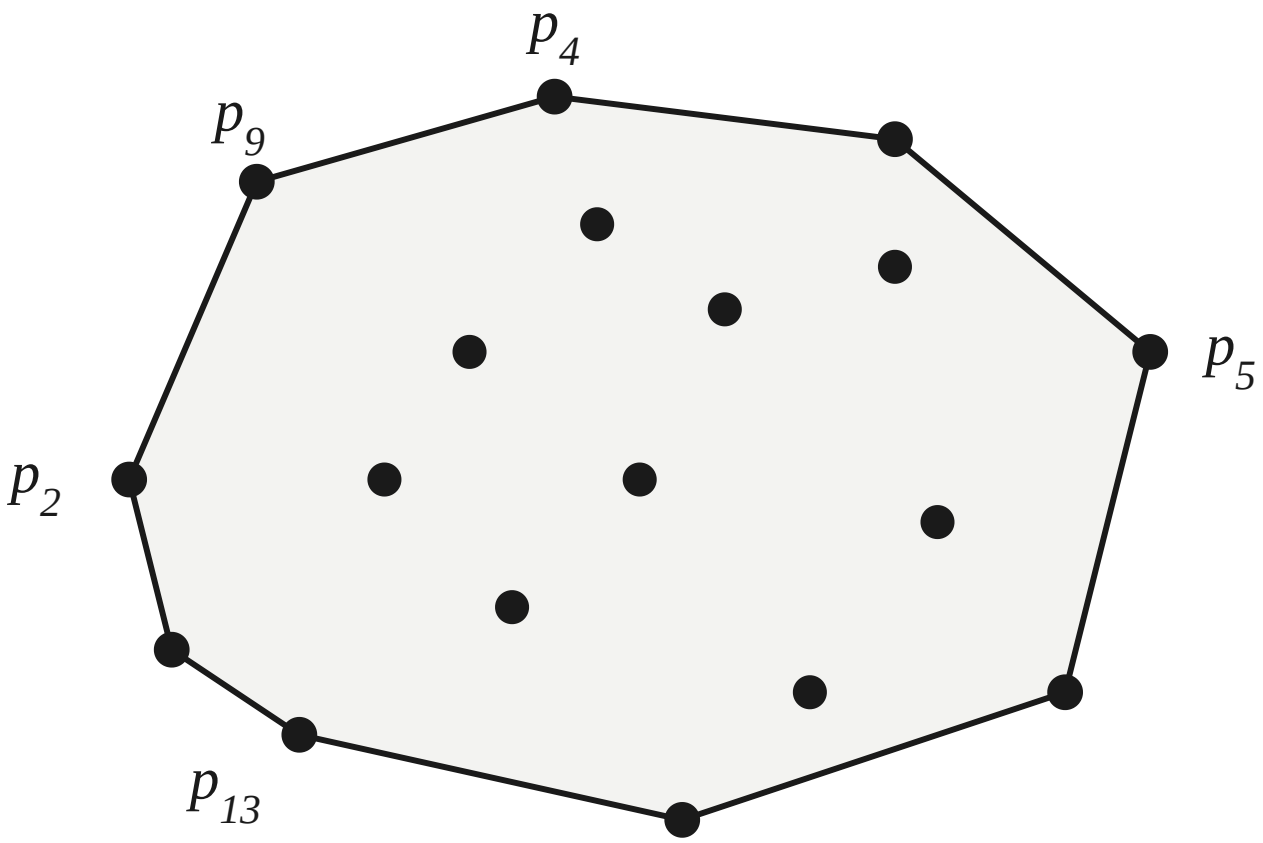
시간 복잡도 성장 비교

같은 n에서 n log n(빨강)은 n^2보다 훨씬 천천히 자란다 — 큰 입력일수록 정렬 알고리즘 선택이 중요해지는 이유.

Convex Hull

점 집합 $P = \{p_1, \ldots, p_n\}, p_i = (x_i, y_i)$ 의 **convex hull**

- P의 모든 점을 포함하는 가장 작은 볼록 다각형

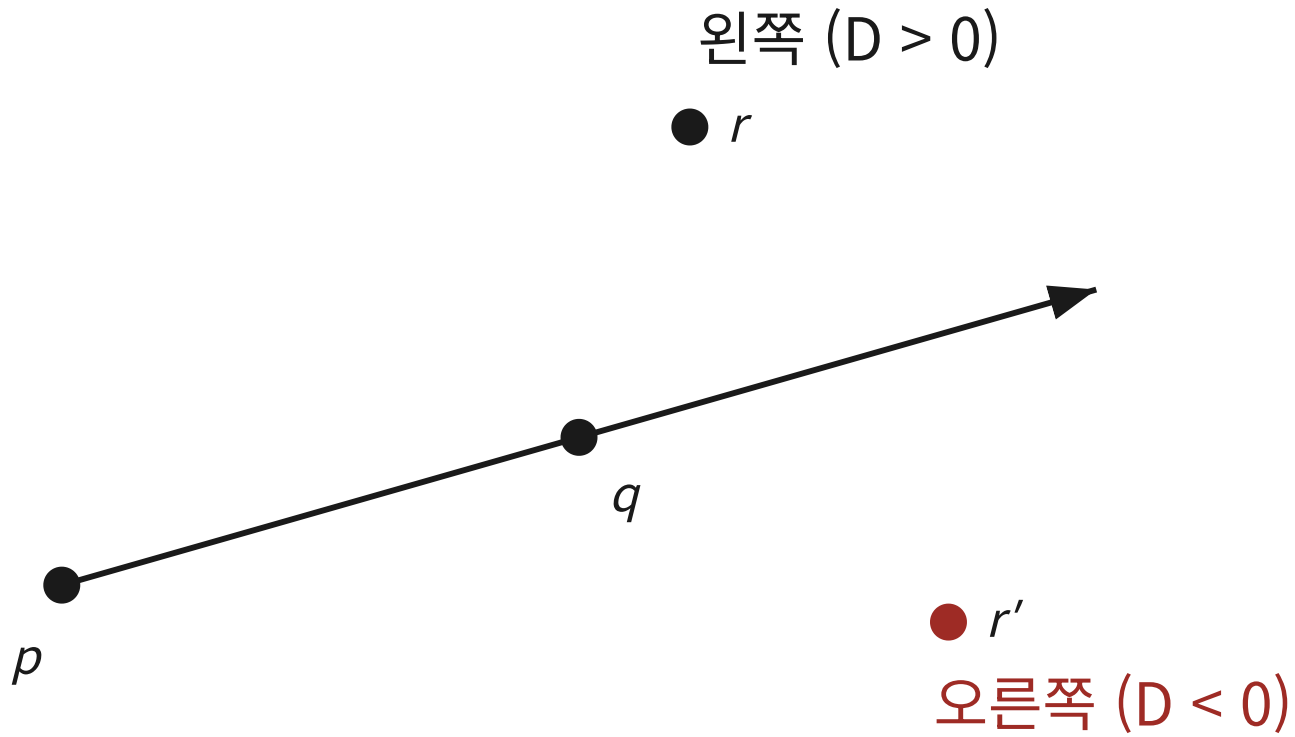


볼록 껍질 — 바깥 점 $p_2, p_9, p_4, \ldots, p_5, \ldots, p_{13}$이 꼭짓점, 안쪽 점은 버려짐

좌/우 판정 (orientation test)

r is left of $\vec{pq} \iff (r_y - p_y)(q_x - p_x) > (q_y - p_y)(r_x - p_x)$

- $D = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x)$ 로 두면 (두 벡터 $\vec{pq}$, $\vec{pr}$의 determinant)
- $D > 0 \to$ 왼쪽 · $D < 0 \to$ 오른쪽 · $D = 0 \to$ 세 점이 한 직선
- 판정 1회 = 곱셈 2번 + 뺄셈 몇 번 = $O(1)$



방향 $p \to q$로 걸을 때 r이 왼쪽(D > 0) / 오른쪽(D < 0)

판정 예시 — $p = (0, 0), q = (4, 2)$

r	$(r_y - p_y)(q_x - p_x)$	$(q_y - p_y)(r_x - p_x)$	**판정**
(2, 3)	12	4	D = 12 − 4 = 8 > 0 → 왼쪽
(2, 0)	0	4	D = 0 − 4 = −4 < 0 → 오른쪽
(6, 3)	12	12	D = 12 − 12 = 0 → 세 점이 한 직선

Brute force: $O(n^3)$

- 아이디어: 점 쌍 (p, q)마다 **나머지 점이 전부 $\vec{pq}$ 오른쪽**인지 검사 → 그러면 변 pq는 hull의 변 (두 방향을 모두 검사하므로, hull의 변은 나머지 점이 전부 오른쪽에 놓이는 방향으로 발견된다. 세 점이 한 직선 위에 있지 않다고 가정한다)
- 후보 쌍 C(n, 2) = n(n−1)/2개 × 검사 $O(n)$ → $T(n) = O(n^3)$

n	$\binom{n}{2}$	**나머지 점 수 (n−2)**	**후보 × 검사 ≈ n^3**	$n \log_2 n$

10	45	8	720	33.2
100	4,950	98	970,200	664
1,000	499,500	998	997,002,000	9,966

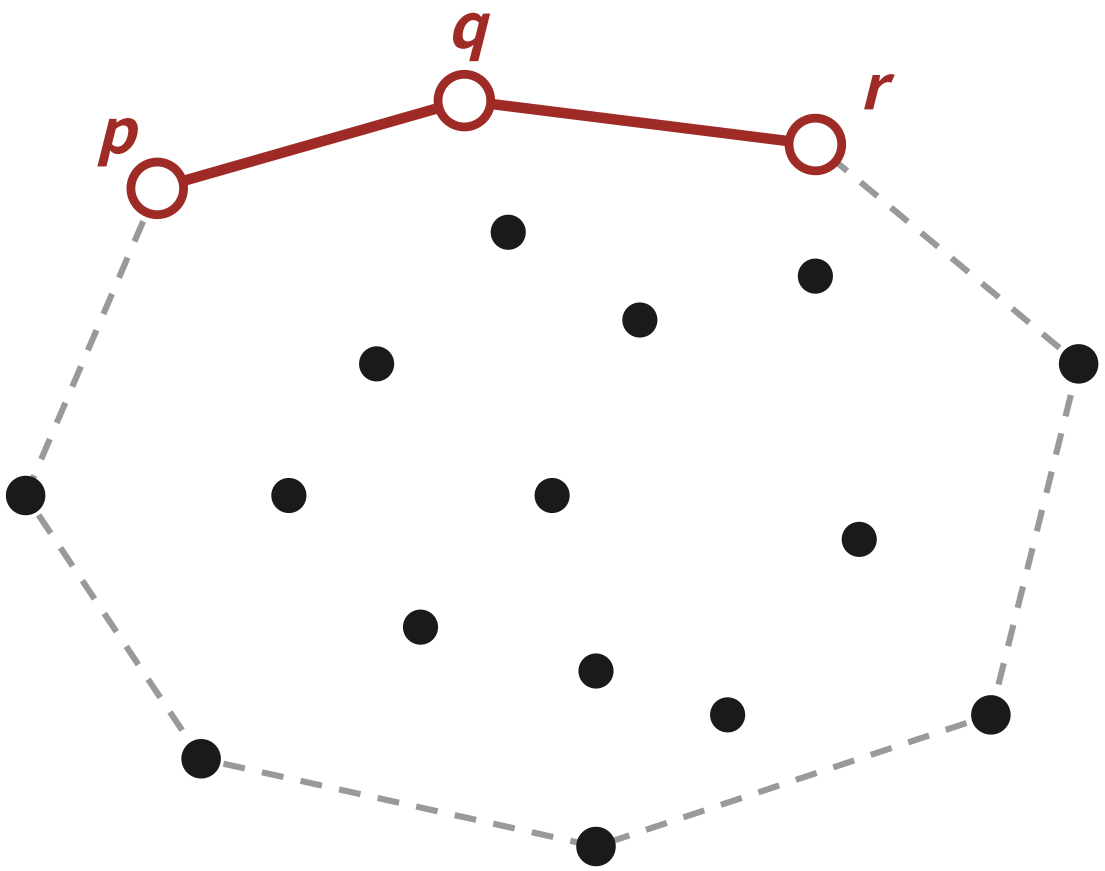
양방향 검사 기준으로 n=1000에서 brute-force의 검사 횟수는 997,002,000회이고, n $\log_2$ n은 약 9,966이므로 두 수치의 비율은 약 10만 배이다. 입력 크기가 커질수록 두 알고리즘의 연산 횟수 차이가 크게 벌어지므로 더 효율적인 알고리즘이 중요하다.

Graham's scan: $O(n \log n)$

가정: 모든 점의 x 좌표는 서로 다르고, 세 점이 한 직선 위에 있지 않다. 점들을 x 좌표 순으로 정렬한 리스트 L 위에서, 이어진 세 점 (p, q, r)을 보며 두 규칙만 반복 (구현 절차는 monotone chain 방식)

- **규칙 (1)** — r이 $\vec{pq}$ **오른쪽** → 한 칸 앞으로 (새 $(q, r, \cdot)$ 검사)
- **규칙 (2)** — 아니면(**왼쪽**) → q를 L에서 **삭제**하고 가능하면 한 칸 뒤로

왜 삭제해도 되나: D>0인 좌회전이면 x 좌표 순서상 q는 선분 pr보다 아래에 있으므로 현재 upper chain에서 q를 제거한다. 이는 q가 upper chain의 꼭짓점이 될 수 없다는 뜻이지, 전체 convex hull의 꼭짓점이 아니라는 뜻은 아니다. q는 lower chain의 꼭짓점으로 남을 수 있으며, 아래쪽 사슬은 점들을 역순으로 순회하면서 같은 규칙을 적용해 별도로 구성한다.



· 오른쪽 → 규칙 (1): 한 칸 전진 · 왼쪽 → 규칙 (2): q 삭제

r이 $\vec{pq}$ 왼쪽(좌회전) → q 삭제. 점선 = 최종 hull의 나머지 부분

손계산 4 — Graham's scan 추적 (점 8개)

A(0,0) B(1,−2) C(2,3) D(3,1) E(4,4) F(5,−1) G(6,2) H(7,0), 좌우 값 = $(r_y - p_y)(q_x - p_x) - (q_y - p_y)(r_x - p_x)$

단계	(p, q, r)	좌우 값	방향	규칙	규칙 뒤 L
1	(A, B, C)	7	왼쪽	(2) B 삭제	A C D E F G H
2	(A, C, D)	−7	오른쪽	(1) 전진	A C D E F G H
3	(C, D, E)	5	왼쪽	(2) D 삭제	A C E F G H
4	(A, C, E)	−4	오른쪽	(1) 전진	A C E F G H

5	(C, E, F)	−11	오른쪽	(1) 전진	A C E F G H
6	(E, F, G)	8	왼쪽	(2) F 삭제	A C E G H
7	(C, E, G)	−6	오른쪽	(1) 전진	A C E G H
8	(E, G, H)	−2	오른쪽	(1) 전진	A C E G H

- 결과 위쪽 사슬: (A, C, E, G, H)
- 이 스캔은 위쪽 사슬만 만든다. 전체 hull은 리스트를 역순으로 순회하며 같은 두 규칙을 적용해 아래쪽 사슬을 만든 뒤, 두 사슬을 이어붙여 완성한다.
- 삭제는 점마다 최대 1번 → 규칙 (2) $\leq n-2$회, 전진도 $O(n)$회 → 스캔 전체 $O(n)$

$T(n) = O(n \log n)$(정렬) + $O(n)$(스캔) = $O(n \log n)$

원문 보충: convex hull 가정과 scan 추적

- convex hull은 P를 포함하는 가장 작은 convex set. 같은 말로 꼭짓점이 P의 점인 가장 큰 convex polygon. 점 집합을 경계 다각형으로 인식하는 데 쓴다.
- general position: 세 점이 한 직선 위에 없다고 가정한다. 좌표를 빼면 p 기준의 두 벡터를 얻고, determinant는 두 벡터가 만드는 부호 있는 평행사변형 넓이이다.
- 슬라이드 scan은 위쪽 사슬 하나만 만든다. 아래쪽 사슬도 같은 방식으로 한 번 더 만든다. q 삭제 후 앞쪽 세 점을 다시 검사하기 위해 한 칸 뒤로 간다.
- 점 8개 예의 단계 3에서 D 삭제 후 단계 4는 (A,C,E). 단계 1에서 맨 앞의 B를 삭제하면 뒤로 갈 곳이 없어 r이 C에서 D로 밀린다.
- r은 앞으로만 가므로 규칙 (1)은 n−2번 이하, 예에서는 5번. 규칙 (2)는 삭제마다 점이 줄어들어 n번 이하, 예에서는 3번.
- 슬라이드의 삭제 횟수 n−h는 전체 hull 기준이다. 이 upper-chain scan에서는 정확히 n−(남은 점 수) = 8−5 = 3. 전체 hull 꼭짓점 수는 h = 7이지만 어느 쪽이든 O(n) 결론은 같다.

점근 표기 (asymptotic notation): O, Ω, Θ, o, ω

- $T(n)$은 보통 **worst case** 실행 시간 함수다. 다만 Big-O 같은 점근 표기 자체가 worst case를 뜻하지는 않는다 — 어떤 실행 시간 함수를 대상으로 삼았는지에 따라 worst·average·best case 분석이 갈린다

Big-O

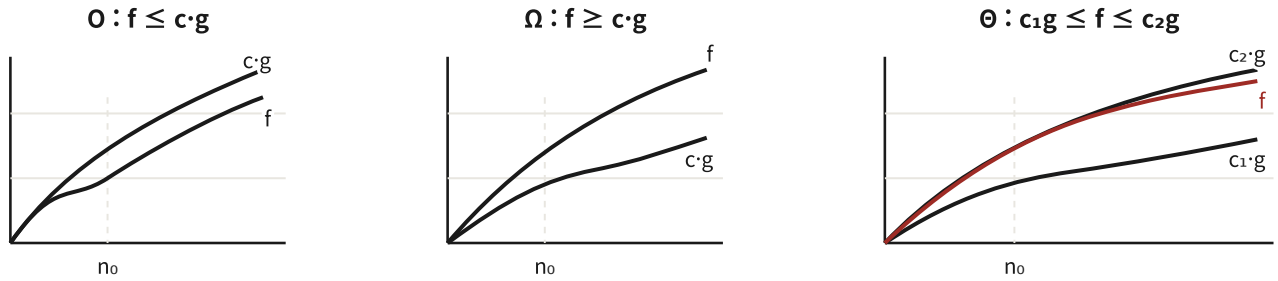
$$f(n) = O(g(n)) \iff \exists c > 0,\ n_0 \geq 0 \ \text{ s.t. } \forall n \geq n_0 :\ f(n) \leq c \cdot g(n)$$

단, f, g는 충분히 큰 n에서 비음수(실행 시간·비용 함수)라고 가정한다.

- 충분히 큰 n에서 f는 g의 어떤 상수배를 넘지 않음 — "g보다 빨리 늘지 않는다"

Ω — 부등호 반대: $f(n) \geq c \cdot g(n)$ — "적어도 g만큼은 든다"

Θ — $f = \Theta(g) \iff f = O(g)$ 이고 $f = \Omega(g)$, 즉 $c_1 g(n) \leq f(n) \leq c_2 g(n)$



O : 충분히 큰 n에서 f ≤ c·g

Ω : 충분히 큰 n에서 f ≥ c·g

Θ : 상한과 하한을 모두 만족

O, Ω, Θ의 그림 — n_0 이후로 f가 $c \cdot g$ 아래 / 위 / 사이에 갇힘

표기	조건	직관	$\lim f/g$
$f = O(g)$	$\exists\, c > 0$: $f \leq cg$	g보다 빨리 늘지 않음	$\neq \infty$
$f = \Omega(g)$	$\exists\, c > 0$: $f \geq cg$	g만큼 이상 빠름	$\neq 0$
$f = \Theta(g)$	$\exists\, c_1, c_2 > 0$: $c_1 g \leq f \leq c_2 g$	같은 속도	$\neq 0, \infty$
$f = o(g)$	$\forall\, c > 0\ \exists\, n_0$: $f \leq cg$	g보다 엄격히 느림	$= 0$
$f = \omega(g)$	$\forall\, c > 0\ \exists\, n_0$: $f \geq cg$	g보다 엄격히 빠름	$= \infty$

단, 극한 $\lim f/g$ 값으로 판정하는 것은 극한이 존재하고 충분히 큰 n에서 g(n)>0일 때만 가능하다. 극한이 존재하지 않으면(진동 등) 정의의 부등식으로 직접 판정한다.

정의 보충: 충분히 큰 n에서 함수들이 비음수이고 g(n) > 0이라는 전제하에 — f = o(g): 모든 c > 0에 대해 어떤 n_0가 존재하여 모든 n ≥ n_0에서 f(n) < cg(n). f = ω(g): 모든 c > 0에 대해 어떤 n_0가 존재하여 모든 n ≥ n_0에서 f(n) ≥ cg(n). lim f(n)/g(n) = L이고 0 < L < ∞이면 f = Θ(g).

- **O, Ω, Θ는 어떤 c 하나만 있으면(∃) 되지만, o, ω는 모든 c에 대해(∀) 성립해야 함** — 시험 단골 포인트
- $f = \omega(g) \iff g = o(f)$

손계산 5 — 상수(c_1, c_2, n_0)직접 찾기

- $14n^2 = \Theta(n^2)$: $c_1 = c_2 = 14, n_0 = 0$
- $f(n) = 14n^2 + n\log_2 n$: $n \geq 1$에서 $n\log_2 n \leq n^2 \to f \leq 15n^2$ $(c_2 = 15)$, $f \geq 14n^2$ $(c_1 = 14) \to \Theta(n^2), n_0 = 1$

n	$14n^2$	$n\log_2 n$	$f(n)$	$15n^2$	$14n^2 \le f \le 15n^2$
1	14	0	14	15	성립
2	56	2	58	60	성립
4	224	8	232	240	성립
8	896	24	920	960	성립
16	3,584	64	3,648	3,840	성립
1024	14,680,064	10,240	14,690,304	15,728,640	성립

손계산 6 — 극한$\lim f/g$로 판정

f	g	$n=10$	$n=100$	$n=1000$	극한	결론
$14n^2$	n^2	14	14	14	14	Θ
$n\log_2 n$	n^2	0.332	0.0664	0.00997	0	o
3^n	n^5	0.590	5.15×10^{37}	1.32×10^{462}	∞	ω
$\log_2 n$	$\log_{10} n$	3.32	3.32	3.32	3.32	Θ

성질과 단순화 규칙

추이성	$f=O(g), g=O(h)\to f=O(h)$ (Ω, Θ도 동일)
합	$f=O(h), g=O(h)\to f+g=O(h)$ — 예: 정렬 $O(n\log n)$ + 스캔 $O(n)$ = $O(n\log n)$
다항식	$a_0+a_1n+\cdots+a_dn^d=\Theta(n^d)$ $(a_d>0)$ — 최고차항만 남음
로그 밑 무관	$\log_a n=\log_b n\,/\,\log_b a\to O(\log_a n)=O(\log_b n)$
성장률 위계	$x>0$이면 $\log n=O(n^x)$ · $r>1, d>0$이면 $n^d=O(r^n)$ — **exponential ≫ polynomial ≫ logarithm**

단순화 4 규칙

곱해진 상수 생략	$14n^2 \to n^2$
차수 높은 항이 압도	n^2이 n을 압도 (n^a, $a > b$)
exponential이 polynomial 압도	3^n이 n^5(와 2^n)를 압도
polynomial이 logarithm 압도	n이 $\log^3 n$, n^2이 $n \log n$을 압도

O-판별은 큰 n 기준 — $100n$ vs n^2:

n	$100n$	n^2	**더 작은 쪽**
10	1,000	100	n^2
100	10,000	10,000	같음
1000	100,000	1,000,000	$100n$

$n = 100$까지는 n^2이 작거나 같지만, 점근적으로는 $100n = O(n)$이 우승

- 극한으로 O, Ω, Θ, o, ω를 판정하는 규칙은 해당 극한이 존재할 때만 쓴다.
- 원문 극한 예: $3^n > n^5$가 이후에도 계속 성립하는 첫 정수는 n = 11.
- FIB1(8) 문제의 추가 질문: 1을 반환하는 잎의 수는 F_8 = 21개.

연습문제

문제 1

FIB1(8)에 대해 (1) 총 호출 수 $T(8)$, (2) 덧셈 횟수, (3) F_2가 호출되는 횟수를 구하라.

힌트 보기

$T(n) = T(n-1) + T(n-2) + 1$ (즉 $T(n) = 2F_{n+1} - 1$), 덧셈 수 = $F_{n+1} - 1$, F_k 호출 수 = F_{n-k+1}.

정답 보기

개념은 위 힌트 참고.

① 점화식 재귀 전개: $T(8) = T(7) + T(6) + 1 = 41 + 25 + 1 = 67$ (표의 $T(7) = 41$, $T(6) = 25$ 사용)

② 덧셈 횟수는 내부 노드 수: $F_9 - 1 = 34 - 1 = 33$

③ F_k 호출 수 공식에 $k = 2, n = 8$: $F_{8-2+1} = F_7 = 13$번

최종 답 $T(8) = 67$ 회 호출, 덧셈 33회, F_2는 13번 호출 (참고: $F_8 = 21$)

문제 2

FIB2(9)를 손으로 추적하라: 배열 $f[0..9]$를 채우고, 덧셈 횟수를 FIB1(9)와 비교하라.

힌트 보기

FIB2는 $f[i] = f[i-1] + f[i-2]$를 $i = 2..n$까지 한 번씩만 계산 — 덧셈 정확히 $n-1$회.

정답 보기

개념은 위 힌트 참고.

① 초기화: $f[0] = 0, f[1] = 1$

② 반복 채우기: 1, 2, 3, 5, 8, 13, 21, 34 → $f[0..9] = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34)$

③ 마지막 칸: $f[9] = f[8] + f[7] = 21 + 13 = 34$

④ 덧셈 횟수 비교: FIB2는 $9 - 1 = 8$회, FIB1(9)는 $F_{10} - 1 = 54$회

최종 답 $f[9] = 34$ · **FIB2 덧셈 8회 vs FIB1 덧셈 54회**

문제 3

$n = 10^9$일 때 $n \log_2 n$과 n^2을 비교하라. ($\log_2 10^9 \approx 30$ 사용)

힌트 보기

비율로 비교: $n^2 / (n \log_2 n) = n / \log_2 n$.

정답 보기

개념은 위 힌트 참고.

① $n \log_2 n \approx 10^9 \times 29.9 = 2.99 \times 10^{10}$

② $n^2 = 10^{18}$ (초당 10^9회 컴퓨터에서도 n^2은 약 31년)

③ 비율: $n^2 / (n \log_2 n) = n / \log_2 n \approx 3.34 \times 10^7$

최종 답 n^2 이 $n \log_2 n$의 약 3,300만 배

문제 4

$f(n) = 6n^2 + n\log_2 n = \Theta(n^2)$임을 보이는 상수 c_1, c_2, n_0를 찾아라.

힌트 보기

$n \geq 1$이면 $\log_2 n \leq n$이고 따라서 $n\log_2 n \leq n^2$.

정답 보기

개념은 위 힌트 참고.

① 아래쪽: $f(n) \geq 6n^2$ (추가 항이 음이 아님) → $c_1 = 6$

② 위쪽: $n \geq 1$에서 $n\log_2 n \leq n^2$이므로 $f(n) \leq 6n^2 + n^2 = 7n^2$ → $c_2 = 7$

③ 부등식이 성립하는 시작점: $n_0 = 1$

④ 검산: $n = 2$ → $24 \leq 26 \leq 28$ ✓ · $n = 8$ → $384 \leq 408 \leq 448$ ✓ · $n = 64$ → $24{,}576 \leq 24{,}960 \leq 28{,}672$ ✓

최종 답 $c_1 = 6,\ c_2 = 7,\ n_0 = 1$ → $6n^2 \leq f(n) \leq 7n^2\ (n \geq 1)$

문제 5

다음 각 쌍에 대해 f가 g의 o, Θ, ω 중 무엇인지 판정하라.
(a) $n^2 \log_2 n$ vs n^3 (b) 3^n vs 2^n (c) $\log_2 n$ vs $\log_{10} n$

힌트 보기

극한 $\lim f/g$로 판정: = 0이면 o, = ∞이면 ω, 0도 ∞도 아닌 상수면 Θ.

정답 보기

개념은 위 힌트 참고.

① (a) $n^2 \log_2 n \,/\, n^3 = (\log_2 n)/n \to 0$ (polynomial이 logarithm을 압도) → o

② (b) $3^n/2^n = (3/2)^n \to \infty$ (밑이 큰 exponential이 압도) → ω

③ (c) $\log_2 n / \log_{10} n = \log_2 10 \approx 3.32$ (밑 변환 상수) → Θ

최종 답 **(a)** o · **(b)** ω · **(c)** Θ

문제 6

점 5개 A, B, C, D, E를 Graham's scan으로 처리한다. 초기 L = A B C D E, 시작 $(p, q, r) = (A, B, C)$이고 각 단계의 좌우 판정 값이 아래 표와 같을 때, **방향 · 적용 규칙 · 규칙 뒤 L**을 채워라.

단계	(p, q, r)	좌우 값	방향	규칙	규칙 뒤 L
1	(A, B, C)	−5	?	?	?
2	(B, C, D)	5	?	?	?
3	(A, B, D)	−5	?	?	?
4	(B, D, E)	−9	?	?	?

점 좌표는 원 노트에 없어 판정 값으로 대체 제공

힌트 보기

양수 = 왼쪽 → 규칙 (2) q 삭제 · 음수 = 오른쪽 → 규칙 (1) 전진. 삭제 후에는 한 칸 뒤로 돌아가 새 삼중을 검사.

정답 보기

개념은 위 힌트 참고.

① (A,B,C) = −5 < 0 → 오른쪽 → 규칙 (1) 전진. L = A B C D E

② (B,C,D) = 5 > 0 → 왼쪽 → 규칙 (2) C 삭제. L = A B D E

③ 삭제했으므로 한 칸 뒤로: (A,B,D) = −5 < 0 → 오른쪽 → 규칙 (1) 전진. L = A B D E

④ (B,D,E) = −9 < 0 → 오른쪽 → 규칙 (1) 전진. L = A B D E, 끝점 도달로 종료

최종 답 **위쪽 사슬 (A, B, D, E) — 전진 $n - 2 = 3$회, 삭제 1회(= 5 − 4단계)**